FLP — Funkcionální a lifetime programování

Projekt v jazyce Rust

Dungeon Map Scribe

Parser a analyzátor map dungeonu v jazyce Rust

Bodová hodnota:	7 bodů
Odevzdání:	archiv `xlogin00.zip`
Jazyk:	Rust (stable, edice 2021)
Externí crates:	Zakázány — pouze `std`

1. Přehled

Vaším úkolem je implementovat backendovou knihovnu pro editor dungeonů ve stylu roguelike her. Návrháři úrovní (levelů) kreslí mapy jako ASCII textové soubory — podobně jako ve starších hrách typu *NetHack* nebo *Rogue* a váš nástroj je musí parsovat, ověřovat jejich strukturu a generovat statistické reporty.

Jedná se o čistě knihovní projekt. Tenká vrstva příkazové řádky je již poskytnuta. Vaším úkolem je implementovat pouze jádro logiky, projít dodanými testy a zajistit, aby byl kód napsán idiomaticky v Rustu (kód není jen funkční, ale je napsaný způsobem, který je v Rust komunitě považovaný za přirozený, čitelný, bezpečný a dobře udržovatelný).

1.1 Formát mapy

Mapa dungeonu je obdélníková mřížka znaků. Každý znak reprezentuje jeden typ dlaždice:

Znak	Typ dlaždice	Význam
`#`	Wall	Pevná zeď — blokuje pohyb
`.`	Floor	Otevřená podlaha — průchozí
`@`	PlayerStart	Počáteční pozice hráče — vyžaduje se právě jedna
`E`	Enemy	Nepřítel — nula nebo více
`T`	Treasure	Truhla s pokladem — nula nebo více
`X`	Exit	Portál pro odchod — vyžaduje se alespoň jeden
`+`	Door	Dveře — průchozí
`^`	Trap	Podlahová past — průchozí

Příklad mapy:

```
##########
#@..T....#
#.######.#
#..E...E.#
####.#####
#...E..T.#
#.......X#
##########
```

1.2 Co musí vaše knihovna umět

- Parsovat víceřádkový `&str` do struktury `DungeonMap`.
- Ověřit, že je mapa validní (správná struktura a přítomnost povinných dlaždic).
- Spočítat, kolik dlaždic každého typu mapa obsahuje.
- Najít pozice všech dlaždic daného typu.
- Vypočítat, kolik průchozích dlaždic (průchozí dlaždice jsou takové dlaždice, na které je možné vstoupit, teda všechny kromě `Wall`) je dosažitelných z dané startovní pozice (flood fill).
- Vygenerovat formátovaný textový report shrnující výsledky.

1.3 Studijní cíle

Tento projekt vám má dát praktickou zkušenost s oblastmi Rustu, ve kterých tento jazyk skutečně vyniká. Po dokončení byste měli umět:

- parsovat strukturovaný text do datového modelu s využitím typu `Result` a vlastních chybových typů,
- navrhovat `enum` typy, které výstižně modelují doménové koncepty i chyby,
- správně pracovat s principy ownershipu a borrowingu při návrhu API nad vlastněnými datovými strukturami,
- skládat složitější posloupnosti operací nad iterátory s využitím `map`, `filter_map`, `flat_map`, `enumerate` a `collect`
- implementovat flood-fill algoritmus idiomaticky v Rustu,
- generovat deterministický a formátovaný textový výstup ze strukturovaných dat,
- psát unit i integrační testy pro čisté funkce.

2. Co je již poskytnuto

Stáhněte si ze systému Moodle připravený skeleton projektu. Stažený skeleton projektu je už v základní podobě zkompilovatelný — všechny studentské funkce obsahují zástupné `todo!()` implementace. Neměňte žádný soubor označený jako **POSKYTNUTO**.

`Soubor`	Stav	Obsah
`src/main.rs`	POSKYTNUTO	CLI wrapper — načte cestu k souboru, zavolá vaši knihovnu a vytiskne report. Neupravujte.
`src/lib.rs`	POSKYTNUTO	Reexportuje všechny veřejné typy. Neupravujte.
`src/error.rs`	POSKYTNUTO	`ParseError` a `ValidationError` včetně všech variant a implementace `Display`. Neupravujte varianty ani `Display`.
`src/tile.rs`	IMPLEMENTUJTE	Enum `Tile` (varianty jsou deklarované). Implementujete `from_char` a `is_passable`.
`src/model.rs`	IMPLEMENTUJTE	Struktura `DungeonMap` a typ `Position`. Implementujete všechny metody.
`src/analysis.rs`	IMPLEMENTUJTE	Funkce `reachable_floor_size`. Implementujete tělo.
`src/report.rs`	IMPLEMENTUJTE	Funkce `generate_report`. Implementujete tělo.
`tests/*.rs`	POSKYTNUTO	Základní integrační testy (`integration_tests.rs`). Vlastní testovací soubor přidejte podle úkolu 11.
`maps/*.txt`	POSKYTNUTO	Pět ukázkových map pro ruční testování.

2.1 Poskytnuté typy — tyto neměňte

Následující typy jsou deklarované ve skeletonu a jejich struktura je pevně daná.

```
// src/error.rs  (poskytnuto - neměňte)

pub enum ParseError {
    EmptyInput,
    JaggedMap { row: usize, expected: usize, found: usize },
    UnknownTile { c: char, row: usize, col: usize },
}

pub enum ValidationError {
    MissingPlayerStart,
    MultiplePlayerStarts(Vec<Position>),
    NoExit,
    NoFloor,
}

// src/tile.rs  (varianty poskytnuty - implementujete metody)

#[derive(Debug, Clone, Copy, PartialEq, Eq, Hash)]
pub enum Tile {
    Wall, Floor, PlayerStart, Enemy,
    Treasure, Exit, Door, Trap,
}

// src/model.rs  (struct poskytnut - implementujete všechny metody)

pub type Position = (usize, usize);  // (řádek, sloupec)

pub struct DungeonMap {
    tiles: Vec<Vec<Tile>>,
    width: usize,
    height: usize,
}
```

3. Implementační úkoly

Implementujte všechny níže uvedené funkce. U každé je zadána hlavička funkce a tělo `todo!()`. Nesmíte měnit žádnou veřejnou hlavičku funkce.

Úkol 1 — `Tile::from_char`

```
pub fn from_char(c: char) -> Result<Tile, ParseError>
```

Převeďte jeden znak na odpovídající variantu `Tile`. Pro jakýkoliv znak, který není uveden v tabulce výše, vraťte `ParseError::UnknownTile`. Pole `row` a `col` v této chybě mají být při přímém volání této metody nastavena na `0`; správné pozice doplní až `DungeonMap::parse`.

- Všech 8 rozpoznaných znaků se musí mapovat na správnou variantu.
- Jakýkoli nerozpoznaný znak (včetně mezery, číslic nebo malých písmen) musí vrátit `Err`.

Úkol 2 — `Tile::is_passable`

```
pub fn is_passable(&self) -> bool
```

Vrací `true` pro každý typ dlaždice, který není `Wall`. Tato metoda je využita při flood-fill algoritmu.

Úkol 3 — `DungeonMap::parse`

```
pub fn parse(input: &str) -> Result<DungeonMap, ParseError>
```

Parsujte víceřádkový `&str` do `DungeonMap`. Jde o nejdůležitější parsovací funkci v celém projektu.

- Vraťte `ParseError::EmptyInput`, pokud je vstup prázdný nebo obsahuje pouze bílé znaky a nové řádky.
- Odstraňujte `\r` z konců řádků, aby byly akceptovány jak unixové, tak windowsové textové soubory.
- Každý znak parsujte pomocí `Tile::from_char` a do případné chyby doplňte správné `row` a `col`.
- Vraťte `ParseError::JaggedMap { row, expected, found }`, pokud má některý řádek jinou délku než první řádek.
- Při úspěchu správně nastavte `width` a `height`.

Nápověda: Idiomatický přístup v Rustu je použít `lines().enumerate()` a potom `chars().enumerate()` spolu s `map` a `collect::<Result<Vec<_>, _>>()`. Tento vzor přirozeně propaguje chyby při sestavování `Vec<Vec<Tile>>`.

Úkol 4 — `DungeonMap::get`

```
pub fn get(&self, pos: Position) -> Option<&Tile>
```

Vraťte referenci na dlaždici na pozici `(row, col)`, nebo `None`, pokud je pozice mimo hranice mapy. Vrácená reference si půjčuje data z `&self`.

Úkol 5 — `DungeonMap::rows`

```
pub fn rows(&self) -> impl Iterator<Item = &Vec<Tile>>
```

Vraťte iterator přes řádky mapy. Každá položka je reference na jeden řádek (`Vec<Tile>`). Tato metoda se používá ve funkcích `count_tiles` a `find_all`.

Poznámka: Metoda vracející `impl Iterator` vyžaduje pochopení toho, že iterator si půjčuje data z `&self`. Jde o záměrné cvičení.

Úkol 6 — `DungeonMap::count_tiles`

```
pub fn count_tiles(&self) -> HashMap<Tile, usize>
```

Vraťte `HashMap`, která mapuje každou variantu `Tile` na počet jejích výskytů v mapě. Zahrňte pouze dlaždice, které se vyskytují alespoň jednou — chybějící dlaždice se nesmí vyskytovat jako klíče s hodnotou 0.

Nápověda: `self.rows().flat_map(|row| row.iter())` vám dá plochý iterator přes všechny dlaždice v mapě. Využijte jej.

Úkol 7 — `DungeonMap::find_all`

```
pub fn find_all(&self, tile: Tile) -> Vec<Position>
```

Vraťte `Vec<Position>` se všemi pozicemi, kde se daný typ dlaždice vyskytuje. Výsledek musí být seřazen v row-major pořadí: nejprve podle řádku, potom podle sloupce.

- Pokud se daná dlaždice v mapě nevyskytuje, vraťte prázdný `Vec`.
- Pro sestavení pozic použijte `enumerate` jak na iteratoru řádků, tak na iteratoru sloupců.

Úkol 8 — `DungeonMap::validate`

```
pub fn validate(&self) -> Result<(), Vec<ValidationError>>
```

Zkontrolujte strukturální validitu mapy. Na rozdíl od většiny funkcí vracejících `Result` tato funkce musí nasbírat všechny chyby a vrátit je najednou — nesmíte skončit u první nalezené chyby.

- Vraťte `Ok(())` právě tehdy, když je mapa plně validní.
- Vraťte `Err(errors)`, kde `errors` je `Vec` obsahující všechny relevantní varianty `ValidationError`.

Kontrolují se tyto čtyři podmínky:

- `MissingPlayerStart` — mapa neobsahuje žádný `PlayerStart`,
- `MultiplePlayerStarts(positions)` — mapa obsahuje více než jeden `PlayerStart`. `Vec` musí obsahovat všechny jejich pozice,
- `NoExit` — mapa neobsahuje žádný `Exit`,
- `NoFloor` — mapa neobsahuje žádný `Floor`.

Důležité: Tato funkce záměrně nemůže používat operátor `?` pro předčasný návrat. Musíte postupně sestavit `Vec<ValidationError>`, přidávat chyby při kontrole jednotlivých podmínek a na konci vrátit `Err(errors)`, pokud vektor není prázdný.

Úkol 9 — `analysis::reachable_floor_size`

```
pub fn reachable_floor_size(map: &DungeonMap, start: Position) -> usize
```

Vraťte počet průchozích dlaždic dosažitelných ze startovní pozice pomocí čtyřsměrného pohybu (nahoru, dolů, doleva, doprava), aniž by bylo možné procházet zdmi.

- Vraťte 0, pokud je start mimo hranice mapy.
- Vraťte 0, pokud je dlaždice na startovní pozici `Wall`.
- Započítejte i samotnou startovní dlaždici.
- K rozhodnutí, zda je na dlaždici možné vstoupit, používejte `is_passable()`.
- Očekává se standardní BFS nebo DFS implementace — použijte `VecDeque` nebo `Vec` jako frontier.
- Navštívené pozice sledujte pomocí `HashSet<Position>`.

Toto je algoritmicky nejzajímavější úkol. Přistupujte k němu pečlivě. `Position` je `(usize, usize)`, tedy `Copy`, takže pozice můžete ukládat do fronty bez problémů s lifetimes. Dávejte pozor na odčítání od `usize` — odčítání od nuly povede k `panic`. Chraňte se explicitně pomocí podmínek typu `row == 0` a `col == 0`.

Úkol 10 — `report::generate_report`

```
pub fn generate_report(map: &DungeonMap) -> String
```

Vygenerujte kompletní souhrnný report mapy jako `String`. Formát je přesně daný — skryté testy porovnávají výstup znak po znaku.

Požadovaný formát pro validní mapu:

```
=== Dungeon Report ===
Dimensions: 10 x 8
Tiles:
  Wall:          26
  Floor:         38
  PlayerStart:    1
  Enemy:          3
  Treasure:       2
  Exit:           1
Validation: OK
Reachable floor from player: 42
```

Pokud validace selže, nahraďte řádek `Validation` tímto blokem:

```
Validation: FAILED
  - Missing PlayerStart
  - No Exit found
```

Pravidla pro report:

- `Dimensions`: `width x height` (počet sloupců krát počet řádků),
- sekce `Tiles`: zahrňte pouze typy dlaždic, které se vyskytují alespoň jednou, a to v pořadí `Wall`, `Floor`, `PlayerStart`, `Enemy`, `Treasure`, `Exit`, `Door`, `Trap`,
- počty dlaždic jsou zarovnané doprava v poli šířky 4 (např. `{:>4}`),
- jako start pro `Reachable floor` použijte pozici `PlayerStart`; pokud neexistuje, uveďte 0,
- každý řádek končí jediným znakem nového řádku; na konci nesmí být navíc prázdný řádek.

Úkol 11 — Vlastní integrační testy

Vytvořte v adresáři `tests/` nový soubor (například `tests/my_tests.rs`) a napište do něj alespoň 3 vlastní integrační testy. Podrobné požadavky viz sekci 6.4.

4. Pravidla a omezení

4.1 Pevná pravidla

Pravidlo	Důsledek porušení
Žádné `unwrap()` ani `expect()` v knihovním kódu	Viz sekci 8.2.
Žádné externí crates (pouze `std`)	Použití externích crates je v rozporu se zadáním a bude při hodnocení penalizováno.
Žádný `unsafe` kód	Automatická nula za daný úkol.
Neupravujte `main.rs`, `lib.rs` ani deklarace variant v `error.rs`	Odevzdání nemusí projít proti skrytým testům.
Nesmíte měnit veřejné hlavičky funkcí	Skryté testy se nebudou kompilovat.
`cargo fmt` nesmí vracet změny	Kontrolováno v CI; možná penalizace −0,25 bodu.
`cargo clippy - -D warnings` musí projít	Kontrolováno v CI; možná penalizace −0,25 bodu za každou skupinu chyb.

4.2 Pokyny k `unwrap()` a `expect()`

V testovacích funkcích a blocích označených atributem `#[cfg(test)]` můžete `unwrap()` a `expect()` používat bez omezení. To je v testech běžná a idiomatická praxe.

V knihovním kódu mimo testy chyby propagujte pomocí `?` nebo je explicitně zpracujte. Pokud v netestovacím kódu saháte po `unwrap()`, berte to jako signál, že je potřeba návrh řešení přehodnotit.

4.3 Clippy a formátování

Před odevzdáním spusťte a opravte vše, co tyto příkazy hlásí:

```
cargo fmt
cargo clippy -- -D warnings
```

Čistý výstup `clippy` bez varování je součástí hodnocení. Mezi časté problémy v tomto projektu patří:

- použití `for` cyklu tam, kde by byl čitelnější iteratorový adaptér,
- zbytečné volání `clone()`,
- příliš složité `match` výrazy, které by šlo zjednodušit pomocí `if let` nebo `matches!`.

4.4 Struktura kódu

Soukromé pomocné funkce by v tomto projektu neměly být potřeba. Většinu úloh lze vyřešit přímo v připravených funkcích. Nesmíte měnit žádnou veřejnou hlavičku funkce ani přidávat nové veřejné moduly. Veškerá logika musí zůstat v poskytnutých modulech — nevkládejte ji do `main.rs`.

5. Funkční požadavky

Tyto požadavky jsou ověřovány skrytou sadou testů. Každý z nich odpovídá jednomu nebo více implementačním úkolům.

FR-1 — Parsování dlaždic

- Každý rozpoznaný znak se mapuje na správnou variantu `Tile`.
- Nerozpoznané znaky vracejí `ParseError::UnknownTile` obsahující znak a jeho `(row, col)`.
- Znaky `\r` jsou před parsováním odstraněny (Windows konce řádků jsou podporované).

FR-2 — Parsování mapy

- Prázdný vstup nebo vstup obsahující pouze bílé znaky vrátí `ParseError::EmptyInput`.
- Neobdélníkový vstup vrátí `ParseError::JaggedMap` se správným indexem řádku, očekávanou šířkou a skutečně nalezenou šířkou.
- Validní obdélníková mapa vytvoří `DungeonMap` se správně nastavenými hodnotami `width` a `height`.

FR-3 — Počítání dlaždic

- `count_tiles` vrátí `HashMap` s jednou položkou pro každý typ dlaždice, který se vyskytuje alespoň jednou.
- Dlaždice, které se nevyskytují, v mapě vůbec nejsou uvedeny (žádné nulové položky).

FR-4 — Hledání entit

- `find_all` vrací pozice seřazené nejprve podle řádků, potom podle sloupců.
- Pokud se daná dlaždice v mapě nevyskytuje, vrací prázdný `Vec`.

FR-5 — Validace

- Mapa s právě jedním `PlayerStart`, alespoň jedním `Exit` a alespoň jednou `Floor` vrací `Ok(())`.
- Ve výsledném `Vec` musí být zahrnuty všechny relevantní chyby `ValidationError`, nikoli jen první nalezená.
- `MultiplePlayerStarts` musí obsahovat všechny startovní pozice.

FR-6 — Dosažitelnost

- Vrací 0 pro start mimo hranice mapy.
- Vrací 0, pokud je startovní dlaždice `Wall`.
- Správně spočítá všechny průchozí dlaždice v propojené komponentě obsahující start.
- Nezapočítává dlaždice v odpojených regionech.

- Nepřeteče ani nespadne na hraničních pozicích.

FR-7 — Generování reportu

- Report přesně odpovídá předepsanému formátu.
- Typy dlaždic jsou vypsány v kanonickém pořadí: `Wall`, `Floor`, `PlayerStart`, `Enemy`, `Treasure`, `Exit`, `Door`, `Trap`.
- Uvedeny jsou jen dlaždice, které se skutečně vyskytují.
- Počty jsou zarovnané doprava v poli šířky 4.
- Každá validační chyba je na samostatném řádku a začíná dvěma mezerami a pomlčkou.

6. Testování

6.1 Spouštění testů

Celou sadu testů spustíte příkazem:

```
cargo test
```

Pouze integrační testy:

```
cargo test --tests
```

Pouze unit testy uvnitř knihovny:

```
cargo test --lib
```

6.2 Poskytnuté testy

Adresář `tests/` obsahuje integrační testy, které jsou pro vás viditelné. Pokrývají hlavní happy-path scénáře i vybrané chybové případy a měly by vám dát dobrou zpětnou vazbu, zda se ubíráte správným směrem.

Poskytnuté testy nejsou vyčerpávající. To, že všechny projdou, ještě nezaručuje plný počet bodů.

6.3 Skryté testy

Při hodnocení se používá i samostatná sada skrytých testů. Ta pokrývá například:

- mapy s Windows konci řádků (`\r\n`),
- mapy, kde je `PlayerStart` v rohu,
- mapy se dvěma zcela odpojenými oblastmi podlahy,
- mapu obsahující všech 8 typů dlaždic právě jednou,
- přesné porovnání výstupu `generate_report` pro referenční mapu,
- validaci se všemi čtyřmi chybami současně,
- volání `reachable_floor_size` se startem na zdi,
- velmi širokou mapu (1 řádek, mnoho sloupců), která ověří, že si nepletete `width` a `height`.

6.4 Vlastní testy

Integrační test ověřuje součinnost více funkcí najednou — například naparsuje mapu, zavolá `validate()` a zkontroluje výstup `generate_report()` v jednom testu. Za smysluplný test se považuje takový, který ověřuje konkrétní očekávané chování (přesný výsledek, správnou chybu nebo chování na hraničním vstupu) a nepokrývá scénáře již obsažené v dodaných testech. Test, který pouze zavolá funkci a skončí u `unwrap()`, se za smysluplný nepovažuje.

7. Odevzdání

Odevzdejte:

- dokončený projekt jako archiv .zip nahraný do systému Moodle; soubor pojmenujte podle svého školního loginu ve tvaru `xlogin00.zip`, kde `xlogin00` nahraďte vlastním loginem,
- soubor `NOTES.md` v kořenovém adresáři repozitáře (maximálně 300 slov), kde stručně popíšete případná neobvyklá návrhová rozhodnutí, známá omezení nebo předpoklady.

Před odevzdáním si ověřte, že `cargo build`, `cargo test`, `cargo fmt` a `cargo clippy - -D warnings` proběhnou bez chyb.

8. Hodnoticí rubrika

Celkem lze získat 7 bodů. Rozdělení bodů:

#	Kritérium	Body	Co se kontroluje
1	Správnost parsování dlaždic a mapy	1,5	Všech 8 znaků; správné chyby `jagged/empty`; CRLF; správná pozice v `UnknownTile`.
2	Počítání dlaždic a hledání entit	1,0	`count_tiles` správně (chybějící dlaždice nejsou uvedeny); `find_all` správně a seřazeně.
3	Validace	1,5	Kontrola všech čtyř chyb; akumulace všech chyb (bez předčasného návratu); `Ok(())` pro validní mapu.
4	Dosažitelnost / flood fill	1,5	Správný BFS/DFS; zeď jako start vrací 0; bezpečné hraniční pozice; oddělené regiony se nepletou.
5	Generování reportu	0,5	Přesná shoda formátu; správné pořadí dlaždic; správné zarovnání.
6	Vlastní integrační testy	1,0	Nový soubor v `tests/` s alespoň 3 vlastními testy; testy nejsou kopií dodaných testů; každý test obsahuje alespoň jedno `assert`.
Celkem		7,0	

8.1 Částečné hodnocení

Kritéria 1 až 6 jsou hodnocena primárně automatizovanými testy. Částečné body lze získat například takto:

- **Kritérium 1 (1,5 b):** 1,0 bodu, pokud se všechny znaky správně parsují, ale strukturální chyby mají špatný typ; 0,5 bodu, pokud parsování funguje, ale nevalidní vstup vyvolá `panic`.
- **Kritérium 2 (1,0 b):** 0,5 bodu, pokud je správně jen jedna ze dvou funkcí.
- **Kritérium 3 (1,5 b):** 0,75 bodu, pokud jsou kontrolované všechny varianty, ale použijete předčasný návrat.
- **Kritérium 4 (1,5 b):** 1,0 bodu za převážně správné řešení s jednou chybou; 0,5 bodu, pokud funguje základní případ, ale selhávají hraniční pozice nebo odpojené regiony.
- **Kritérium 5 (0,5 b):** 0,25 bodu za drobnou odchylku ve formátování při jinak správném obsahu.
- **Kritérium 6 (1,0 b):** 0,5 bodu za 1–2 vlastní testy splňující podmínky; 0,0 bodu, pokud soubor v `tests/` chybí nebo obsahuje pouze kopie dodaných testů.

8.2 Horní limit při neidiomatickém řešení

Odevzdání, které projde všemi testy, ale používá `unwrap()` nebo `expect()` v knihovním kódu, je omezeno maximálně na 5,5/7,0 bez ohledu na ostatní výsledky.

9. Tipy pro úspěch

Začněte od základů

Nejprve implementujte a otestujte `Tile::from_char` a `DungeonMap::parse`. Na těchto dvou funkcích závisí vše ostatní. Po každé menší změně si spusťte testy.

Implementujte v pořadí

Úkoly jsou v zadání uvedeny v pořadí podle závislostí. Implementace mimo toto pořadí vám pravděpodobně zkomplikuje ladění. Odolejte nutkání začít rovnou s `generate_report`.

Využívejte typový systém

Nechte kompilátor, aby vám řekl, co je špatně. Pokud bojujete s borrow checkerem, obvykle nejde o to, že by borrow checker „neměl pravdu“, ale o signál, že vlastnictví dat není navrženo správně.

Vzor `collect`

Sběr iteratoru `Result` do jediného `Result` je idiomatický Rust a přesně to se hodí pro funkci `parse`:

```
let rows: Result<Vec<Vec<Tile>>, ParseError> =
    lines.map(|line| parse_line(line)).collect();
```

Tento vzor si dobře nastudujte. Umožňuje ukončit celé parsování při první chybě pomocí operátoru `?`, aniž byste museli psát další podmínky navíc.

Flood fill a `usize`

`Position` je `(usize, usize)`. Bezznaménková celá čísla nemohou být záporná. Při generování čtyř sousedů proto vždy nejprve explicitně zkontrolujte `row == 0` a `col == 0`. Nespoléhejte na nepozorné odčítání.

Formátování reportu

Makro `format!()` podporuje modifikátory zarovnání. Pro zarovnání čísla doprava v poli šířky 4 použijte například:

```
format!("  Wall: {:>4}", count)
```

Dodržte formát přesně — skryté testy porovnávají výsledný řetězec znak po znaku.

Příloha — Ukázkové mapy

Tyto mapy jsou poskytnuty v adresáři `maps/` skeletonu. Můžete je použít pro ruční testování například pomocí:

```
cargo run -- maps/simple.txt
```

`maps/simple.txt`

```
##########
#@..T....#
#.######.#
#..E...E.#
####.#####
#...E..T.#
#.......X#
##########
```

Validní mapa. `PlayerStart` je na pozici `(1,1)`. Nepřátelé jsou na pozicích `(3,3)`, `(3,7)` a `(5,4)`. Poklady jsou na `(1,4)` a `(5,7)`. `Exit` je na pozici `(6,8)`.

`maps/complex.txt`

Větší, plně propojená mapa obsahující všech 8 typů dlaždic. Použijte ji pro ověření formátu reportu.

`maps/disconnected.txt`

Mapa se dvěma odpojenými oblastmi podlahy. Hodí se k otestování toho, že `reachable_floor_size` počítá pouze region obsahující hráče, nikoli všechny regiony dohromady.

`maps/invalid_no_player.txt`

Syntakticky validní mapa bez `PlayerStart`. Použijte ji k ověření, že `validate()` vrací `MissingPlayerStart`.

`maps/invalid_jagged.txt`

Mapa, kde je jeden řádek kratší než ostatní. Použijte ji k ověření, že `parse()` vrací `ParseError::JaggedMap` se správným indexem řádku.

10. Poznámka k samostatné práci

Použití generativních jazykových modelů nebudeme v tomto projektu samostatně kontrolovat. Je však na vás, zda zadání využijete hlavně k získání hotového řešení, nebo k pochopení základních konceptů jazyka Rust, které je vhodné zvládnout i bez výrazné externí pomoci. Projekt není natolik rozsáhlý ani obtížný, aby jeho vypracování pomoc jazykového modelu vyžadovalo. Zároveň je záměrně zasazen do herní tematiky, aby byl o něco odlehčenější a zábavnější než čistě technické zadání.